



IL TACCUINO DI DEGUSTAZIONE

Sorso

Lo spazio di *team*, nel tuo sito

GUIDA ALL'INTEGRAZIONE API

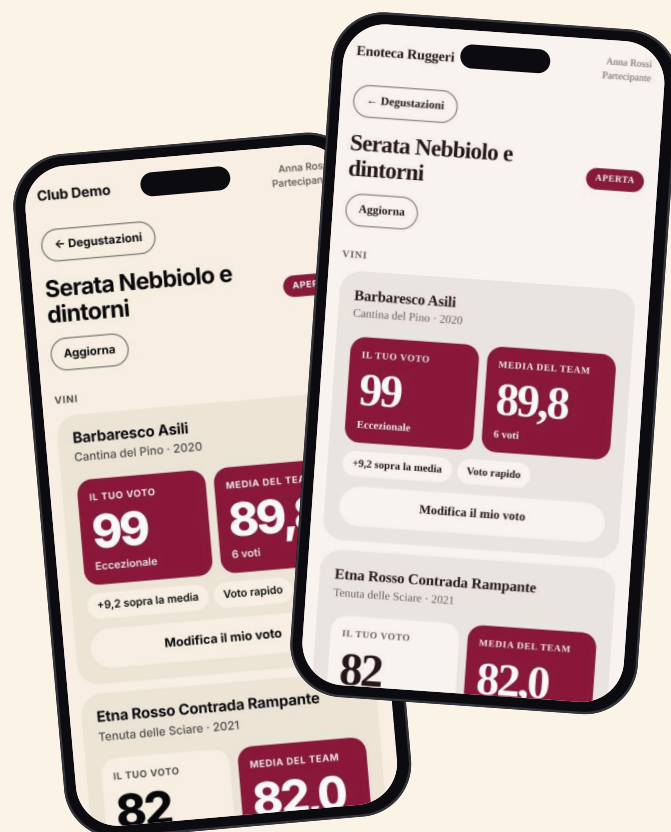
TOKEN FIRMATO · IFRAME · RISULTATI IN SOLA LETTURA

1000

01 IN BREVE

Una degustazione che *vive* nel tuo sito

I tuoi utenti votano insieme i vini di una serata, ognuno vede i propri voti e, solo dopo aver votato, la media del team. Tu incorpori una pagina e firmi un token: al resto pensa Sorso.



1

IFRAME DA
INCORPORARE

1

TOKEN DA FIRMARE
SUL TUO SERVER

50-100

LA SCALA
DEI VOTI

0 €

IL COSTO
DEL SERVIZIO

Per i tuoi utenti

Una pagina elegante e veloce, nei colori del tuo sito: si vota in pochi tocchi, anche dal telefono.

Per chi organizza

Si crea la serata, si aggiungono i vini, si chiude quando è finita: tutto dentro l'iframe.

Sotto il tuo controllo

Accessi, ruoli e gruppi li decidi tu, firmando il token. I risultati li leggi dal tuo server.

02 QUELLO CHE VEDONO GLI UTENTI

Chiaro, veloce, *a misura di telefono*

Dall'elenco delle serate al voto: pochi gesti, numeri grandi, niente da imparare.



1 Con i colori del tuo sito: qui un tema scuro e verde.



2 I vini, con il tuo voto e la media del team dopo aver votato.



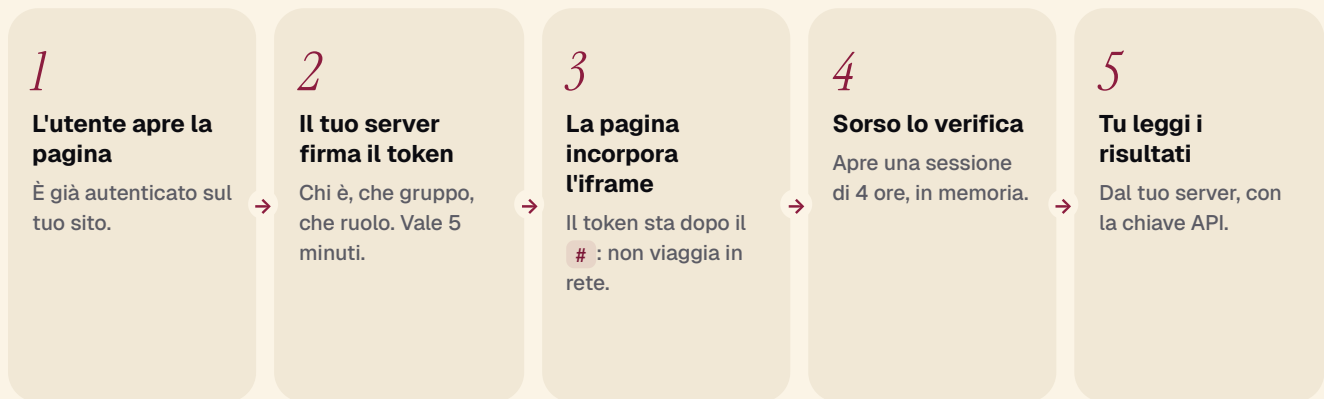
3 Il voto rapido: occhio, naso e bocca da 50 a 100.

«Prima di votare la media resta nascosta: nessuno è influenzato dal voto degli altri.»

03 COME FUNZIONA

Cinque passaggi, *un solo token*

Il tuo sito resta il padrone di casa: decide chi entra, in quale gruppo e con quale ruolo.



Cosa vive *dove*

COSA	DOVE	NOTE
Utenti, password, accessi	Il tuo sito	Sorso non conosce nessuna password.
Gruppi (team) e ruoli	Il tuo token	Li decidi tu a ogni caricamento.
Degustazioni, vini, voti	Sorso	Visibili solo al team; i voti individuali solo a chi li ha dati.
Medie e risultati	Sorso → il tuo server	API di sola lettura, solo aggregati.
Aspetto (colori, logo, titolo)	Sorso, impostato per te	Lo cambiamo noi su tua richiesta.

Nessuna scrittura dall'esterno. Dall'API non si possono scrivere voti: si vota solo dentro l'iframe. Così nessuno può gonfiare una media dal di fuori.

Cosa Sorso *non* fa

Non conosce le password

L'accesso lo gestisce solo il tuo sito: a Sorso arriva un token firmato, mai una credenziale.

Non conserva anagrafiche

Niente email, telefono o indirizzo: solo un identificativo opaco e i voti.

Non mostra i voti altrui

Ognuno vede i propri; la media del team compare solo dopo aver votato quel vino.

04 PRIMA DI COMINCIARE

Tre credenziali e *i tuoi domini*

Te le consegniamo noi dopo la richiesta di attivazione. Il segreto e la chiave ti vengono mostrati **una volta sola**.

CREDENZIALE	A COSA SERVE	DOVE TENERLA
ID partner <code>enoteca-ruggeri</code>	Identifica il tuo sito: va nell'indirizzo dell'iframe e nel claim <code>iss</code> del token.	Non è segreto.
Segreto di firma 64 caratteri	Con questo il tuo server firma il token di ogni utente (HS256).	Solo sul server, in una variabile d'ambiente.
Chiave API <code>sk_</code>	Legge i risultati dal tuo server (sola lettura).	Solo sul server. Se è esposta, chiedi subito la rotazione (vale entro 60 secondi).
Domini autorizzati	I siti da cui si può incorporare lo spazio, con <code>https://</code> e senza percorso.	Da qualunque altro dominio il browser blocca la pagina.

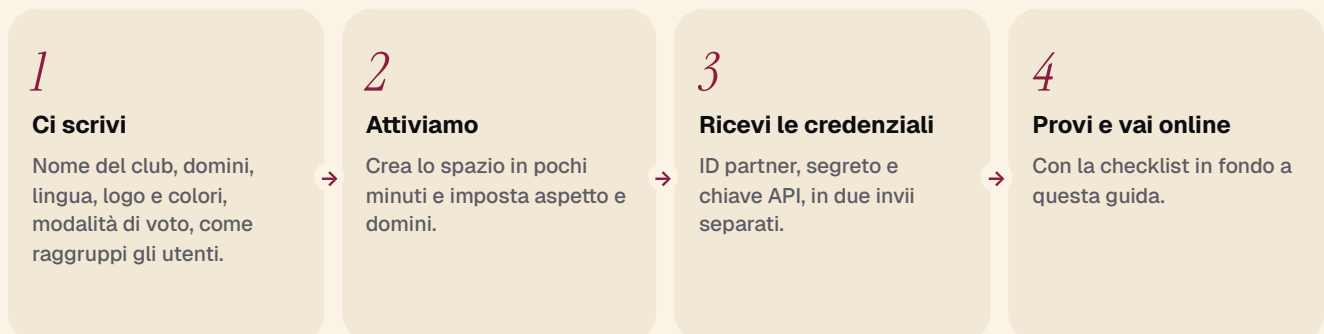
Mai nel browser. Il segreto e la chiave non devono comparire nel codice della pagina, nei log o in un repository. Se li perdi non si possono rileggere: si genera una coppia nuova.

Variabili d'ambiente *consigliate*

```
SORSO_BASE=https://sorso-taccuino.vercel.app
SORSO_PARTNER_ID=enoteca-ruggeri
SORSO_SECRET=...il segreto di firma...
SORSO_API_KEY=sk_enoteca-ruggeri_...
```

SERVER

Come ottenere *l'attivazione*



«Dopo la richiesta, l'attivazione richiede pochi minuti: il tempo vero è quello dell'integrazione nel tuo sito.»

05 PASSO 1 — IL TOKEN

Firma il token *sul tuo server*

Un JWT firmato con HS256 (HMAC-SHA256), generato a ogni caricamento della pagina per l'utente che sta entrando.

CAMPO RICHIESTO SIGNIFICATO E REGOLE

iss	sì	Il tuo ID partner.
sub	sì	L'identificativo dell'utente nel tuo sito: 1–128 caratteri tra lettere, cifre e <code>. _ : @ -</code> . Meglio un id opaco che un'email.
team	sì	Il gruppo: chi ha lo stesso team vede le stesse degustazioni. 1–64 caratteri tra lettere, cifre e <code>_ : -</code> .
jti	sì	Identificativo unico del token (8–64 caratteri): lo rende monouso. Un valore nuovo a ogni token.
exp	sì	Scadenza in secondi Unix, al massimo 15 minuti da adesso. Consigliati 5.
role	no	<code>member</code> (predefinito) oppure <code>organizer</code> .
name	no	Il nome da mostrare (fino a 60 caratteri). Non viene salvato.
lang	no	<code>it</code> oppure <code>en</code> . Se manca vale la lingua del partner.
mode	no	La modalità con cui si apre la scheda di questo utente : <code>smart</code> (voto rapido) o <code>full</code> (scheda completa). Deve essere tra quelle abilitate per il tuo partner, altrimenti è ignorata.

Regole da rispettare. Solo HS256 (qualunque altro algoritmo, compreso `none`, è rifiutato). Il token vale **una volta sola**: generato a ogni costruzione della pagina e non metterlo in cache. Tieni l'orologio del server sincronizzato.

Com'è fatto *un token*

header

L'algoritmo: sempre `HS256`, tipo `JWT`.

payload

Chi è l'utente, in che gruppo, con che ruolo, fino a quando vale: i campi qui sopra.

firma

Calcolata con il tuo segreto: solo tu e Sorso potete produrla.

05 PASSO 1 — ESEMPIO IN NODE.JS

Node.js, *senza librerie*

Usa solo il modulo `crypto` di Node: nessuna dipendenza da installare. Metti la funzione nel codice che costruisce la pagina del tuo sito.

```

// Firma il token per un utente del tuo sito (Node.js, nessuna libreria esterna).
// Da eseguire SUL SERVER del tuo sito, mai nel browser: il segreto non deve uscire di lì.
const crypto = require("crypto");

const PARTNER_ID = process.env.SORSO_PARTNER_ID; // l'ID che ti abbiamo consegnato
const SEGRETO = process.env.SORSO_SECRET; // il segreto di firma (64 caratteri esadecimali)

const b64url = (v) => Buffer.from(v).toString("base64url");

function creaTokenSorso({ sub, name, team, role = "member" }) {
  const header = { alg: "HS256", typ: "JWT" };
  const payload = {
    iss: PARTNER_ID, // chi firma: il tuo ID partner
    sub, // l'ID dell'utente nel TUO sito (meglio un id opaco)
    name, // nome da mostrare, facoltativo
    team, // il gruppo: stesso team = stesse degustazioni
    role, // "member" oppure "organizer"
    jti: crypto.randomBytes(12).toString("hex"), // id unico del token: lo rende monouso
    exp: Math.floor(Date.now() / 1000) + 300, // scade fra 5 minuti
  };
  const corpo = b64url(JSON.stringify(header)) + "." + b64url(JSON.stringify(payload));
  const firma = crypto.createHmac("sha256", SEGRETO).update(corpo).digest("base64url");
  return corpo + "." + firma;
}

// Uso, nel codice che genera la pagina del tuo sito (a ogni caricamento, mai in cache):
// const token = creaTokenSorso({ sub: utente.id, name: utente.nome, team: "giovedì" });
// const url = `https://sorso-taccuino.vercel.app/embed?p=${PARTNER_ID}&token=${token}`;

module.exports = { creaTokenSorso };

if (require.main === module) {
  const [sub, team, role, name] = process.argv.slice(2);
  console.log(creaTokenSorso({ sub, team, role: role || "member", name: name || "" }));
}

```

PROVA DA TERMINALE

```

SORSO_PARTNER_ID=enoteca-ruggeri SORSO_SECRET=...il segreto... \
node firma-token.js utente-42 giovedì organizer "Anna Rossi"

```

Il token stampato si può incollare dopo il `#` dell'indirizzo dell'iframe (passo 2). Genera un token nuovo a ogni prova: ognuno si usa una volta sola.

05 PASSO 1 — ESEMPIO IN PYTHON

Python, *solo libreria standard*

Funziona con Python 3 e la sola libreria standard. Il segreto va letto da una variabile d'ambiente, mai scritto nel codice.

PYTHON

```
# Firma il token per un utente del tuo sito (Python 3, solo libreria standard).
# Da eseguire SUL SERVER del tuo sito, mai nel browser.
import base64, hashlib, hmac, json, os, secrets, sys, time

PARTNER_ID = os.environ["SORSO_PARTNER_ID"] # l'ID che ti abbiamo consegnato
SEGRETO = os.environ["SORSO_SECRET"]       # il segreto di firma (64 caratteri esadecimali)

def b64url(dati: bytes) → str:
    return base64.urlsafe_b64encode(dati).rstrip(b"=").decode()

def crea_token_sorso(sub, team, role="member", name=""):
    header = {"alg": "HS256", "typ": "JWT"}
    payload = {
        "iss": PARTNER_ID,           # chi firma: il tuo ID partner
        "sub": sub,                  # l'ID dell'utente nel TUO sito
        "name": name,                # nome da mostrare, facoltativo
        "team": team,                # il gruppo
        "role": role,                # "member" oppure "organizer"
        "jti": secrets.token_hex(12), # identificativo unico: rende il token monouso
        "exp": int(time.time()) + 300, # scade fra 5 minuti
    }
    corpo = (b64url(json.dumps(header, separators=(",", ":")).encode()) + "."
             + b64url(json.dumps(payload, separators=(",", ":")).encode()))
    firma = hmac.new(SEGRETO.encode(), corpo.encode(), hashlib.sha256).digest()
    return corpo + "." + b64url(firma)

if __name__ == "__main__":
    sub, team = sys.argv[1], sys.argv[2]
    role = sys.argv[3] if len(sys.argv) > 3 else "member"
    name = sys.argv[4] if len(sys.argv) > 4 else ""
    print(crea_token_sorso(sub, team, role, name))
```

PROVA DA TERMINALE

SHELL

```
SORSO_PARTNER_ID=enoteca-ruggeri SORSO_SECRET=_il segreto_ \
python3 firma-token.py utente-42 giovedì organizer "Anna Rossi"
```

Il token stampato si può incollare dopo il `#` dell'indirizzo dell'iframe (passo 2). Genera un token nuovo a ogni prova: ognuno si usa una volta sola.

05 PASSO 1 — ESEMPIO IN PHP

PHP, *nessuna libreria*

Richiede PHP 7.4 o successivo e nessuna libreria. Da eseguire sul server, mai nel browser dell'utente.

```
<?php
// Firma il token per un utente del tuo sito (PHP 7.4 o successivo, nessuna libreria).
// Da eseguire SUL SERVER del tuo sito, mai nel browser.

function base64url(string $dati): string {
    return rtrim(strtr(base64_encode($dati), '+/', '-_'), '=');
}

function creaTokenSorso(
    string $partnerId, string $segreto, string $sub, string $team,
    string $role = 'member', string $name = ''
): string {
    $header = ['alg' => 'HS256', 'typ' => 'JWT'];
    $payload = [
        'iss' => $partnerId,           // chi firma: il tuo ID partner
        'sub' => $sub,                 // l'ID dell'utente nel TUO sito
        'name' => $name,               // nome da mostrare, facoltativo
        'team' => $team,               // il gruppo
        'role' => $role,               // "member" oppure "organizer"
        'jti' => bin2hex(random_bytes(12)), // id unico: rende il token monouso
        'exp' => time() + 300,         // scade fra 5 minuti
    ];
    $corpo = base64url(json_encode($header)) . '.' . base64url(json_encode($payload));
    $firma = hash_hmac('sha256', $corpo, $segreto, true);
    return $corpo . '.' . base64url($firma);
}

// Prova da riga di comando: php firma-token.php utente team [ruolo] [nome]
if (PHP_SAPI === 'cli' && realpath($_SERVER['SCRIPT_FILENAME']) ?? '' === __FILE__) {
    [, $sub, $team, $role, $name] = array_pad($argv, 5, '');
    echo creaTokenSorso(
        getenv('SORSO_PARTNER_ID'), getenv('SORSO_SECRET'),
        $sub, $team, $role ?: 'member', $name
    ), "\n";
}
```

PHP

PROVA DA TERMINALE

```
SORSO_PARTNER_ID=enoteca-ruggeri SORSO_SECRET=...il segreto... \
php firma-token.php utente-42 giovedì organizer "Anna Rossi"
```

SHELL

Il token stampato si può incollare dopo il `#` dell'indirizzo dell'iframe (passo 2). Genera un token nuovo a ogni prova: ognuno si usa una volta sola.

06 PASSO 2 — INCORPORARE LO SPAZIO

Un iframe, *il token dopo il #*

L'indirizzo ha questa forma. Il frammento (dopo il `#`) non viene mai inviato a nessun server: niente token nei log, niente token nell'intestazione Referer.

```
https://orso-taccuino.vercel.app/embed?p=ID_PARTNER#token=TOKEN_FIRMATO
```

INDIRIZZO

```
<!-- La pagina del TUO sito che incorpora lo spazio di team. -->
<!-- L'indirizzo dell'iframe, con il token DOPO il #, lo genera il tuo server a ogni caricamento della pagina. -->
<!-- Il frammento (#token=...) non viene mai inviato a nessun server: non finisce nei log né nel Referer. -->
<iframe id="orso" title="Degustazioni del club"
  src="https://orso-taccuino.vercel.app/embed?p=ID_PARTNER#token=TOKEN_FIRMATO_DAL_TUO_SERVER"
  width="100%" height="700" style="border:0; max-width:640px"
  allow="" referrerpolicy="no-referrer"></iframe>

<script>
  const ORIGINE_ORSO = "https://orso-taccuino.vercel.app";
  const frame = document.getElementById("orso");
  window.addEventListener("message", (e) => {
    if (e.origin !== ORIGINE_ORSO) return; // ignora i messaggi di altri
    const d = e.data || {};
    // Facoltativo: l'iframe comunica la propria altezza, così eviti la barra di scorrimento.
    if (d.type === "orso:height") frame.style.height = d.height + "px";
    // Consigliato: se l'accesso scade (o il token non è più valido), l'iframe chiede un token nuovo.
    // Qui il tuo sito ne chiede uno fresco al proprio server e ricarica l'iframe.
    if (d.type === "orso:reauth") {
      fetch("/il-tuo-endpoint/orso-token", { credentials: "same-origin" }) // il TUO server firma un token
      nuovo
        .then((r) => r.json())
        .then((j) => { frame.src = "https://orso-taccuino.vercel.app/embed?p=ID_PARTNER#token=" + j.token; });
    }
  });
</script>
```

HTML

COSA SUCCEDDE DENTRO L'IFRAME

- ◆ Sorso scambia il token con una **sessione di 4 ore**, in memoria, e poi toglie il token dall'indirizzo.
- ◆ Se la sessione finisce, l'iframe chiede al tuo sito un token nuovo con `orso:reauth`: l'utente rientra da solo e il voto che stava compilando non va perso.
- ◆ Il tasto Indietro del browser resta dentro lo spazio.

MESSAGGI VERSO LA TUA PAGINA

MESSAGGIO	QUANDO
<code>orso:height</code>	L'altezza del contenuto, per adattare l'iframe senza barre di scorrimento.
<code>orso:reauth</code>	L'accesso non è più valido: serve un token nuovo.

Controlla sempre `event.origin` prima di fidarti di un messaggio.

07 RUOLI, TEAM E VISIBILITÀ

Il ruolo lo decidi *tu*

Sorso non ha schermate di amministrazione né inviti: se un utente è organizzatore lo dice il tuo sito, nel claim **role**.

AZIONE	PARTECIPANTE	ORGANIZZATORE
Vedere le degustazioni del team	sì	sì
Creare ed eliminare una degustazione	no	sì
Aggiungere vini	no	sì
Chiudere o riaprire	no	sì
Creare una degustazione alla cieca e svelarla	no	sì
Votare (anche più volte: l'ultimo vale)	sì	sì
Scrivere un'ipotesi (alla cieca)	sì	sì
Vedere la media del team	dopo aver votato quel vino, o a serata chiusa (dal 2° voto)	come il partecipante
Vedere quanti hanno votato	solo dopo aver votato	sempre
Vedere i vini di una cieca non svelata	no: «Vino 1», «Vino 2»...	sì

- ◆ **Degustazione chiusa:** i voti sono definitivi; nessuno può più votare né aggiungere vini. L'organizzatore può riapirla.
- ◆ **Team separati:** un team non vede mai le degustazioni di un altro. Lo stesso **sub** in due team ha dati separati.
- ◆ **Limiti:** 200 degustazioni per team e 100 vini per degustazione; nomi fino a 80 (serata) e 100 (vino) caratteri; note personali fino a 500.



- ★ Lo stesso vino visto da un organizzatore.

08 VOTI E PUNTEGGI

Da 50 a 100, *sempre*

Un vino non prende mai 30, e quasi mai 100. Due modalità, stessa scala: la media le mescola correttamente.

50 INSUFFICIENTE	60 SUFFICIENTE	70 DISCRETO	80 BUONO	90 ECCELLENTE	96-100 ECCEZIONALE · IRRIPETIBILE
---------------------	-------------------	----------------	-------------	------------------	---

Voto *rapido*

Tre giudizi da 50 a 100: occhio (peso 10%), naso (30%), bocca (60%). Veloce, senza descrivere il vino.

Scheda *completa*

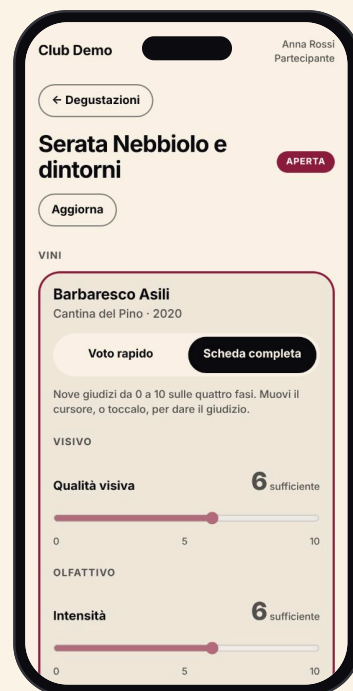
Nove giudizi da 0 a 10 sulle quattro fasi (visivo 10%, olfattivo 30%, gusto-olfattivo 40%, finale 20%). Ogni fase segue una curva e il totale è la media pesata delle fasi.

- ◆ Il punteggio lo calcola sempre il server: quello mostrato mentre si vota è un'anteprima.
- ◆ Il **100** si dà solo se ogni giudizio è al massimo.
- ◆ Ogni numero finale segue l'arrotondamento classico: da 5 in su si sale.

Modalità iniziale. Il partner sceglie quali modalità consentire e quale proporre per prima. Con il claim **mode** puoi deciderla utente per utente; l'utente può comunque cambiarla voto per voto.



1 Voto rapido.

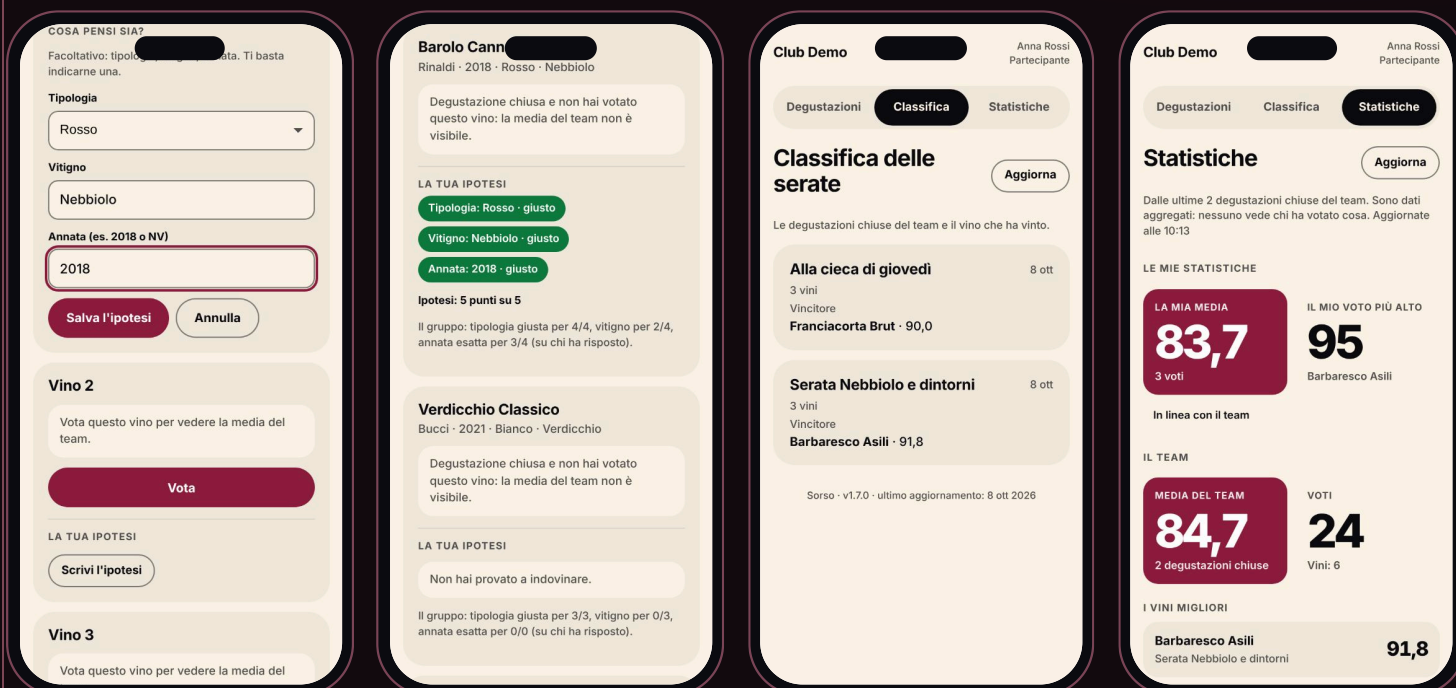


2 Scheda completa.

09 CLASSIFICA, STATISTICHE E ALLA CIECA

Non solo il voto: *tre sezioni in più*

Lo spazio di team ha tre sezioni in alto: **Degustazioni**, **Classifica** e **Statistiche**. L'«alla cieca» è un modo di creare la degustazione.



- 1 Alla cieca:** «Vino 1», e l'ipotesi su tipologia, vitigno, annata.
- 2 Svelata:** i nomi, il punteggio dell'ipotesi, il gruppo.
- 3 Classifica:** le serate chiuse e il vincitore.
- 4 Statistiche:** le mie e quelle del team.

ALLA CIECA

- ◆ L'organizzatore spunta «Alla cieca» creando la degustazione e può indicare, per ogni vino, tipologia e vitigno.
- ◆ Ai partecipanti il server **non manda** nome, produttore, annata, tipologia né vitigno: si vede «Vino N» finché l'organizzatore non preme «Svela i vini».
- ◆ Ognuno vota e può scrivere un'ipotesi. Dopo lo svelamento la degustazione si chiude per sempre: ognuno vede il proprio punteggio (tipologia 1 punto, vitigno 2, annata 2 se esatta, 1 se ±1) e del gruppo solo un riepilogo anonimo.

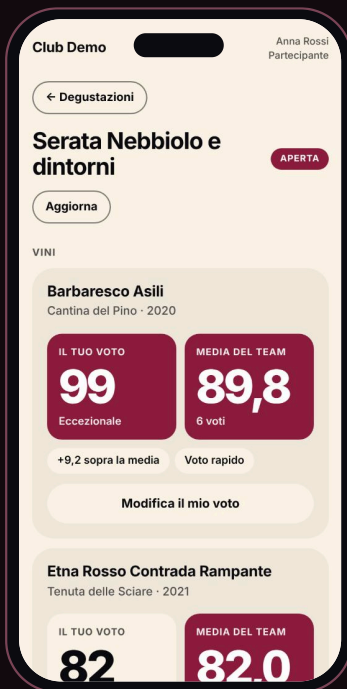
CLASSIFICA E STATISTICHE

- ◆ **Classifica:** nella degustazione, i vini ordinati per media (pari merito compresi); nella sezione Classifica, le serate chiuse con il vincitore.
- ◆ **Statistiche:** dalle ultime 30 degustazioni chiuse del team, con medie, distribuzione, vini migliori, tipologie e, per ognuno, i propri numeri.
- ◆ A degustazione chiusa la media di un vino la vedono tutti, ma solo dal **secondo voto**. Nessuno vede mai chi ha votato cosa.

10 PERSONALIZZAZIONE

Nei *tuo*i colori

Con pochi valori lo spazio prende il tono del tuo sito. Li impostiamo noi, su tua richiesta.



1 **Tema di base:** i colori del vino, senza nessuna scelta.



2 **Enoteca Ruggeri:** accento vinaccia, sfondo carta, carattere con grazie.



3 **Club del Bosco:** accento verde, sfondo scuro, carattere arrotondato.

IMPOSTAZIONE	VALORI	EFFETTO
title	testo, fino a 40 caratteri	Titolo in alto.
logo	indirizzo <code>https</code> di un'immagine	Logo in alto, alto 28 px.
accent	colore <code>#rrggbb</code>	Pulsanti, selezioni, media del team, voti dal 96.
bg · ink	colori <code>#rrggbb</code>	Sfondo e testo.
font	sistema, serif, arrotondato, mono	Carattere (solo tra quelli di sistema).
lingua · modalità	it/en · rapido/completa/entrambe	Lingua predefinita e modalità consentite.

Leggibilità garantita. Se i colori scelti non assicurano un testo leggibile (contrasto almeno 4,5 a 1), Sorso li corregge o torna a quelli predefiniti, anche sui riquadri e sui bordi dei campi. Imposta sempre insieme sfondo e testo. Se non imposti nulla, lo spazio usa i colori del vino: crema e bordeaux, e nero con ciliegia quando il dispositivo è in tema scuro.

// API DI SOLA LETTURA

I risultati, *dal tuo server*

Tutte le richieste vanno fatte **dal server**, mai dal browser: la chiave non deve uscire dal tuo sistema.

COSA	VALORE
Indirizzo di base	<code>https://sorso-taccuino.vercel.app/api/v1</code>
Autenticazione	<code>Authorization: Bearer sk_ID_PARTNER_...</code>
Formato	JSON UTF-8. Ogni risposta porta <code>X-Sorso-API-Version: 1</code> e <code>Cache-Control: no-store</code> .
Dati	Solo aggregati: mai chi ha votato cosa.

Elenco delle *degustazioni*

GET /api/v1/tastings?team=giovedi&status=closed

GET

```
{
  "tastings": [
    {
      "id": "4529a7e3f3",
      "team": "giovedi",
      "name": "Serata di ottobre",
      "status": "closed",
      "blind": false,
      "revealed": false,
      "createdAt": "2026-10-08T10:06:36.581Z"
    }
  ]
}
```

I filtri `team` e `status` (`open` o `closed`) sono facoltativi. Con tanti team conviene sempre filtrare per team.

Risultati di una *degustazione*

GET /api/v1/tastings/{id}/results

GET

```
{
  "tasting": {
    "id": "4529a7e3f3",
    "team": "giovedi",
    "name": "Serata di ottobre",
    "status": "closed",
    "blind": false,
    "revealed": false,
    "createdAt": "2026-10-08T10:06:36.581Z"
  },
  "minVotes": 2,
  "wines": [
    {
      ...
    }
  ]
}
```

// API DI SOLA LETTURA

Come leggere *i risultati*

- ◆ **votes** è il numero di voti; **average** la media del team, con un decimale.
- ◆ La media compare **dal secondo voto** (**minVotes** = 2): con un voto solo coinciderebbe con quello di una persona. Finché mancano voti, **average** è **null** e **hidden** è **true**. **position** è l'ordine del vino; **rank** la posizione in classifica (1, 1, 3: i pari merito); **type** e **grape** sono quelli indicati dall'organizzatore.
- ◆ Una degustazione **alla cieca non svelata** ha **blind: true**, **revealed: false** e **name**, **producer**, **vintage**, **type**, **grape** a **null**: l'API non rivela i vini che i tuoi utenti non devono ancora conoscere.
- ◆ Voti rapidi e schede complete sono sulla stessa scala 50–100: la media li mescola correttamente.

Un limite di ogni media. In un gruppo molto piccolo (2–3 persone), chi vede la media prima e dopo un nuovo voto può dedurre quel voto. Se per te è un problema, mostra i risultati solo a degustazione chiusa.

Esportazione in *CSV*

```
GET /api/v1/tastings/{id}/results?format=csv
```

CSV

```
tasting_id,tasting,wine_id,producer,name,vintage,votes,average,type,grape,rank
4529a7e3f3,Serata di ottobre,e135640f56,Poderi Rivalta,Barolo Cannubi,2019,3,88,,1
4529a7e3f3,Serata di ottobre,9839911744,Tenuta delle Sciare,Etna Rosso Contrada Rampante,2021,2,74.5,,2
4529a7e3f3,Serata di ottobre,07b1c4c96f,Casa Marchigiana,Verdicchio Classico,2022,1,,,,
```

Le celle che iniziano con **= + - @** vengono precedute da un apice: un nome di vino non può essere eseguito come formula.

Cancellare i dati di *un utente*

```
DELETE /api/v1/users/{sub}
```

DELETE

```
{
  "votesRemoved": 2,
  "guessesRemoved": 0
}
```

Rimuove tutti i voti e le ipotesi alla cieca dell'utente (il **sub** del token) e aggiorna le medie. Pensato per le richieste di cancellazione; si può ripetere senza effetti e funziona sempre, anche quando il servizio è in sola lettura.

Provalo in *un minuto*

Dal terminale del tuo server, con la chiave API. Se ricevi un elenco (anche vuoto) la chiave funziona; con **401** è sbagliata o è stata ruotata.

```
curl -sS -H 'Authorization: Bearer sk_ID_PARTNER_...' \
https://sorso-taccuino.vercel.app/api/v1/tastings
```

SHELL

// API DI SOLA LETTURA — ESEMPI

Quattro comandi, *copia e incolla*

SHELL

```
#!/bin/sh
# Esempi dell'API di sola lettura. Da eseguire dal SERVER del tuo sito.
# SORSO_BASE      indirizzo di Sorso, per esempio https://sorso-taccuino.vercel.app
# SORSO_API_KEY   la chiave API che ti abbiamo consegnato (inizia con sk_)

# 1. Elenco delle degustazioni (filtri facoltativi: team, status=open|closed)
curl -sS -H "Authorization: Bearer $SORSO_API_KEY" \
  "$SORSO_BASE/api/v1/tastings?status=closed"

# 2. Risultati di una degustazione (sostituisci ID_DEGUSTAZIONE con un id dell'elenco)
curl -sS -H "Authorization: Bearer $SORSO_API_KEY" \
  "$SORSO_BASE/api/v1/tastings/ID_DEGUSTAZIONE/results"

# 3. Gli stessi risultati in CSV, da aprire in un foglio di calcolo
curl -sS -H "Authorization: Bearer $SORSO_API_KEY" \
  "$SORSO_BASE/api/v1/tastings/ID_DEGUSTAZIONE/results?format=csv" -o risultati.csv

# 4. Cancellare i dati di un utente: usa l'ID che hai messo nel claim "sub"
curl -sS -X DELETE -H "Authorization: Bearer $SORSO_API_KEY" \
  "$SORSO_BASE/api/v1/users/ID_UTENTE"

# 5. Statistiche del team (serve il parametro team: lo stesso valore del claim "team")
curl -sS -H "Authorization: Bearer $SORSO_API_KEY" \
  "$SORSO_BASE/api/v1/stats?team=ID_TEAM"

# 6. Classifica delle serate chiuse del team, con il vino vincitore
curl -sS -H "Authorization: Bearer $SORSO_API_KEY" \
  "$SORSO_BASE/api/v1/events?team=ID_TEAM"

# 7. Riepilogo anonimo delle ipotesi di una degustazione alla cieca già svelata
curl -sS -H "Authorization: Bearer $SORSO_API_KEY" \
  "$SORSO_BASE/api/v1/tastings/ID_DEGUSTAZIONE/guesses"
```

Esegui sempre queste chiamate **dal tuo server**, mai da una pagina del browser: la chiave API non deve essere visibile a nessuno.

// API DI SOLA LETTURA — STATISTICHE E SERATE

Statistiche e *classifica delle serate*

Statistiche *del team*

GET /api/v1/stats?team=giovedi

GET

```
{
  "team": "giovedi",
  "at": "2026-10-08T10:06:36.961Z",
  "window": {"tastings":2, "closedTotal":2, "open":0},
  "totals": {"tastings":2, "open":0, "closed":2, "wines":5, "votes":12},
  "average": 85,
  "top": [ ... ],
  "distribution": [ ... ],
  "byType": [ ... ],
  "events": [ ... ]
}
```

Il parametro **team** è obbligatorio (lo stesso valore del claim **team**). Si basa sulle ultime **30 degustazioni chiuse** e si aggiorna al massimo ogni due minuti: **at** dice quando è stata calcolata. Contiene il numero di degustazioni, vini e voti; la media del team; **top** (i cinque vini migliori); **distribution** (quanti vini per fascia di media); **byType** (media per tipologia) ed **events**. Nessun dato personale.

Le serate *e il vincitore*

GET /api/v1/events?team=giovedi

GET

```
{
  "team": "giovedi",
  "events": [
    {
      "id": "...",
      "name": "Alla cieca di giovedì",
      "createdAt": "2026-10-08T10:06:36.744Z",
      "blind": true,
      "revealed": true,
      "wines": 2,
      "votes": 6,
      "average": 87,
      "winner": {"name": "Barolo Cannubi", "average":91, "votes":3, "tie":false}
    }
  ]
}
```

Le degustazioni chiuse del team, dalla più recente, ciascuna con il vino vincitore (**winner**, con **tie: true** se c'è un pari merito). Servono almeno due voti: altrimenti **winner** è **null**. Per una cieca non svelata il vincitore non ha nome (**hidden: true**).

II API DI SOLA LETTURA — IPOTESI ALLA CIECA

Le ipotesi *alla cieca*

GET /api/v1/tastings/{id}/guesses

GET

```
{
  "tasting": { "id": "...", "revealed": true, ... },
  "wines": [
    {
      "id": "...",
      "position": 1,
      "name": "Barolo Cannubi",
      "type": "Rosso",
      "grape": "Nebbiolo",
      "vintage": "2019",
      "guessers": 3,
      "answers": { "type": { "answered": 3, "correct": 3 }, "grape": { "answered": 3, "correct": 2 }, "year": { "answered": 3,
"exact": 1, "close": 2 } }
    }
  ]
}
```

Solo per le degustazioni alla cieca **già svelate**: prima risponde `409 not_revealed`, e per una degustazione normale `409 not_blind`. Dice, per ogni vino, quante persone hanno provato a indovinare e quante hanno risposto giusto su tipologia, vitigno e annata (`exact` esatta, `close` con un anno di scarto). Mai chi ha detto cosa. Con meno di due persone che hanno risposto, `answers` è `null`: un riepilogo di una persona sola svelerebbe le sue risposte.

Cosa vede *l'API di una cieca*

GET /api/v1/tastings/{id}/results

(cieca non svelata)

GET

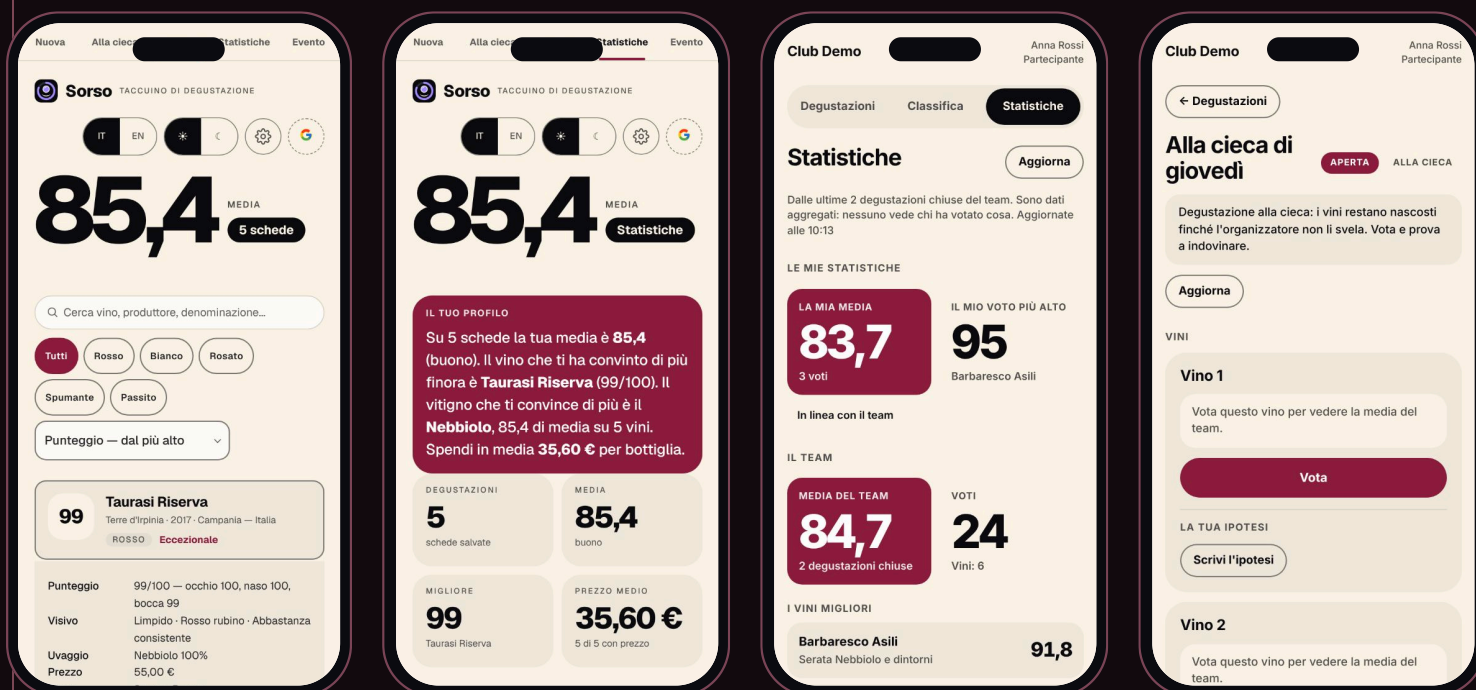
```
{
  "id": "...",
  "position": 1,
  "producer": null,
  "name": null,
  "vintage": null,
  "type": null,
  "grape": null,
  "votes": 3,
  "average": 91,
  "hidden": false,
  "rank": 1
}
```

Finché la degustazione non è svelata, `name`, `producer`, `vintage`, `type` e `grape` sono `null`, anche nel CSV: il tuo sito non può rivelare per sbaglio il vino ai partecipanti. Le medie e la classifica (`rank`) ci sono comunque.

12 APP PERSONALE E SPAZIO DI TEAM

Taccuino, Statistiche, Evento, Cieca: *dove stanno*

L'app personale e lo spazio di team che incorpori hanno Statistiche, classifica e alla cieca; l'API serve il secondo.



- A** Taccuino: l'archivio personale. **B** Statistiche personali. **C** Statistiche del team. **D** Alla cieca nel team.

NELL'APP PERSONALE

Taccuino — l'archivio privato di ogni scheda, con foto, viaggio, descrittori e note.

Statistiche — medie, radar delle fasi, prezzo, andamento nel tempo.

Evento — classifica di gruppo di una serata.

Alla cieca — si assaggia senza etichetta, si prova a indovinare e poi si svela.

NELLO SPAZIO DI TEAM E NELL'API

Nessun archivio personale: ognuno ritrova le **degustazioni del team** e il proprio voto su ogni vino.

Statistiche del team (medie, distribuzione, vini migliori, tipologie) e le proprie di ogni utente; nell'API `/stats`, senza dati personali.

Classifica: dei vini di ogni degustazione e delle serate chiuse con il vincitore; nell'API `rank`, `/events`.

Alla cieca di gruppo: nomi nascosti finché l'organizzatore non svela, ipotesi di tipologia, vitigno e annata, riepilogo anonimo (`/guesses`).

Fuori dall'iframe restano archivio personale, foto, viaggio e prezzo: se ti servono, scrivici.

13 ERRORI E LIMITI

Se qualcosa *non va*

COSA VEDI	CAUSA PROBABILE	COSA FARE
Iframe vuoto o errore del browser	Il dominio della pagina non è tra quelli registrati («Refused to frame»).	Comunicaci il dominio esatto, con <code>https://</code> .
«L'accesso non è valido o è scaduto»	Token con firma sbagliata, scaduto o con campi non validi.	Controlla <code>iss</code> , <code>sub</code> , <code>team</code> , <code>jti</code> , <code>exp</code> e il segreto.
«Questo accesso è già stato usato»	Lo stesso token è arrivato due volte (pagina in cache, antepima automatica).	Un token nuovo a ogni caricamento; niente cache.
«Devi rientrare»	Sessione di 4 ore finita.	Rispondi a <code>sorso:reauth</code> con un token nuovo.
Manca «Nuova degustazione»	L'utente è <code>member</code> .	<code>role: "organizer"</code> nel token.

I codici *dell'API*

HTTP	CODICE	SIGNIFICATO
401	<code>unauthorized</code>	Chiave mancante o non valida.
404	<code>not_found</code>	Degustazione o percorso inesistente (anche di un altro partner).
405	<code>method_not_allowed</code>	Metodo non previsto per quel percorso.
429	<code>rate_limited</code>	Troppe richieste: <code>Retry-After</code> dice quando riprovare.
400 · 409	<code>invalid_team</code> · <code>not_blind</code> · <code>not_revealed</code>	Manca <code>team</code> in <code>/stats</code> e <code>/events</code> ; oppure <code>/guesses</code> su una degustazione normale o non ancora svelata.
503	<code>no_database</code>	Il database non è raggiungibile o non è collegato: riprova fra poco. Se il limite mensile è quasi esaurito, le risposte portano l'intestazione <code>X-Sorso-Quota</code> ; letture e cancellazione dati restano sempre disponibili.

Limiti e *disponibilità*

COSA	LIMITE
Aperture di sessione (<code>/embed</code>)	200 al minuto per indirizzo IP
Operazioni di un utente nell'iframe	90 al minuto per utente
API di sola lettura	120 richieste al minuto per chiave e per IP

Sessione dell'iframe 4 ore; token al massimo 15 minuti, monouso. Servizio gratuito, senza garanzia di continuità: scarica il CSV dopo ogni serata. All'80% dell'uso mensile compare `X-Sorso-Quota: warn`; oltre il 90% si passa in sola lettura fino al mese dopo.

14 SICUREZZA E PRIVACY

Pochi dati, *ben custoditi*

COSA DEVI FARE TU

- ◆ Tieni segreto di firma e chiave API solo sul server; se li esponi, chiedi la rotazione.
- ◆ Firma token solo per utenti già autenticati; team e ruolo vengono dai tuoi dati, mai da parametri della richiesta.
- ◆ Usa come `sub` un identificativo opaco invece dell'email.
- ◆ Servi la pagina che incorpora l'iframe sempre in `https`.

COSA FA SORSO

- ◆ La pagina si incorpora solo dai domini registrati, senza script inline e senza inviare il riferimento di provenienza.
- ◆ Il punteggio lo calcola il server; i voti individuali sono visibili solo a chi li ha dati.
- ◆ Chi ha il token non esce dal proprio team né dal proprio partner: ogni richiesta lo verifica.

Quali dati *conserviamo*

DATO	CONSERVATO	NOTE
Identificativo utente (<code>sub</code>)	sì	Serve a collegare l'utente ai suoi voti. Nessun'altra anagrafica.
Team, nomi di degustazioni e vini	sì	Inseriti dagli organizzatori.
Voti, giudizi, note e ipotesi alla cieca	sì	Visibili solo all'autore; l'API e il gruppo vedono solo medie e riepiloghi anonimi.
Nome visualizzato (<code>name</code>)	no	Resta nella sessione in memoria.
Email, telefono, indirizzo IP	no	L'IP serve solo al limite di richieste, in un contatore temporaneo.

- ◆ I dati restano finché non vengono cancellati. Per cancellare un utente (voti e ipotesi): `DELETE /api/v1/users/{sub}`. Il riepilogo anonimo di una cieca già svelata non cambia.
- ◆ Chi gestisce il servizio ha accesso tecnico al database. Per informativa e basi giuridiche valgono le valutazioni del tuo sito.

15 PRIMA DI ANDARE ONLINE

Spunta ogni voce *con una prova vera*

Non a memoria: apri la pagina, vota, guarda i risultati. Per qualunque dubbio o richiesta (dominio, colori, rotazione dei segreti, cancellazioni) scrivici.

- ☐ Ho ricevuto ID partner, segreto e chiave API e li ho salvati solo sul server.
- ☐ Ho comunicato i domini esatti e l'iframe si apre da quelli; da un dominio non registrato viene bloccato.
- ☐ Il token è generato a ogni caricamento e la pagina non è in cache.
- ☐ Un `member` vota e, solo dopo, vede la media del team.
- ☐ Un `organizer` crea una degustazione, aggiunge vini e la chiude.
- ☐ Una degustazione alla cieca: i partecipanti vedono «Vino 1», «Vino 2»; dopo «Svela i vini» compaiono i nomi e la classifica.
- ☐ Le sezioni Classifica e Statistiche si aprono e mostrano i dati delle degustazioni chiuse.
- ☐ Due utenti di team diversi non vedono le rispettive degustazioni.
- ☐ Ricaricando solo l'iframe l'utente rientra (`soiso:reauth`) o ricarica la pagina.
- ☐ La lettura dei risultati dal server funziona e il CSV si apre correttamente.
- ☐ Ho provato la cancellazione di un utente di prova e le medie si sono aggiornate.
- ☐ Ho deciso chi, da noi, è organizzatore e dove scarichiamo il CSV al termine delle serate.
- ☐ L'informativa agli utenti del nostro sito è aggiornata.

«Il token è monouso e dura pochi minuti: se qualcosa non si apre, quasi sempre è lì.»

Pronti.

Versione 1.7.0 della guida · ultimo aggiornamento: 8 ottobre 2026